Homework 6 - Charlie Caravati

In [9]:
```
import numpy as np
from numpy.linalg import inv
from numpy.linalg import norm

import math
```

1.

(a)

In [10]:
```
def evalF(x):

    F = np.zeros(2)

    F[0] = 3*x[0]**2 + 4*x[1]**2 - 1
    F[1] = x[1]**3 - 8*x[0]**3 - 1

    return F
```

$$G(x) = \begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} 0.016 & -0.17 \\ 0.52 & -0.26 \end{bmatrix} \begin{bmatrix} 3x^2 + 4y^2 - 1 \\ y^3 - 8x^3 - 1 \end{bmatrix}$$

In [19]:
```
def evalG(x):

    A = np.array([[0.016, -0.17],
         [0.52, -0.26]])

    G = x - A @ evalF(x)

    return G
```

In [18]:
```
def fixedSys(x0,tol,Nmax):

    ''' inputs: x0 = initial guess, tol = tolerance, Nmax = max its'''
    ''' Outputs: xstar= approx root, ier = error message, its = num its'''

    for its in range(Nmax):

       x1 = evalG(x0)

       if (norm(x1-x0) < tol):
           xstar = x1
           ier =0
           return[xstar, ier, its]

       x0 = x1

    xstar = x1
    ier = 1

    #print(f"Root: {xstar} \n Error: {ier} \n Iterations: {its}")
    return[xstar,ier,its]
```

In [28]:
```
# (i) (x0, y0) = (-0.5, 0.25)

eps = 10e-4

x0 = [-0.5, 0.25]

fixedSys(x0, eps, 100)
```

Out[28]: [array([-0.49725479, 0.25407792]), 0, 1]

In [26]:
```
# (ii) (x0, y0) = (-1.5, 1.25)

eps = 10e-4

x0_ii = [-1.5, 1.25]

fixedSys(x0_ii, eps, 100)
```

```
/var/folders/nm/zhg7jf3n4gz76nqmv2nbzvrr0000gn/T/ipykernel_95326/1657570400.py:6: RuntimeWarning: overflow encountered in scalar power
  F[1] = x[1]**3 - 8*x[0]**3 - 1
/var/folders/nm/zhg7jf3n4gz76nqmv2nbzvrr0000gn/T/ipykernel_95326/1657570400.py:6: RuntimeWarning: invalid value encountered in scalar subtract
  F[1] = x[1]**3 - 8*x[0]**3 - 1
```

Out[26]: [array([nan, nan]), 1, 99]

In [22]:
```
# (iii) (x0, y0) = (-3, 3)

eps = 10e-4

x0_iii = [-3, 3]

fixedSys(x0_iii, eps, 100)
```

```
/var/folders/nm/zhg7jf3n4gz76nqmv2nbzvrr0000gn/T/ipykernel_95326/1657570400.py:6: RuntimeWarning: overflow encountered in scalar power
  F[1] = x[1]**3 - 8*x[0]**3 - 1
/var/folders/nm/zhg7jf3n4gz76nqmv2nbzvrr0000gn/T/ipykernel_95326/1657570400.py:6: RuntimeWarning: invalid value encountered in scalar subtract
  F[1] = x[1]**3 - 8*x[0]**3 - 1
```

Out[22]: [array([nan, nan]), 1, 99]

> The iteration for the first initial guess (i) converges to the root in one iteration because the true root [-0.49725479, 0.25407792] is so close to the initial [-0.5, .25] starting point. The iterations for guesses (ii) and (iii) diverge and trigger overflow warnings as the initial guess is outside the basin of convergence.

(b)

In [31]:
```
def evalJ(x):


    J = np.array([[6*x[0], 8*x[1]],
        [-24*x[0]**2, 3*x[1]**2]])


    return J
```

In [30]:
```
def Newton(x0,tol,Nmax):

    ''' inputs: x0 = initial guess, tol = tolerance, Nmax = max its'''
    ''' Outputs: xstar= approx root, ier = error message, its = num its'''

    for its in range(Nmax):
       J = evalJ(x0)
       Jinv = inv(J)
       F = evalF(x0)

       x1 = x0 - Jinv.dot(F)

       if (norm(x1-x0) < tol):
           xstar = x1
           ier =0
           return[xstar, ier, its]

       x0 = x1

    xstar = x1
    ier = 1
    return[xstar,ier,its]
```

In [32]:
```
# (i) (x0, y0) = (-0.5, 0.25)

eps = 10e-4

x0 = [-0.5, 0.25]

Newton(x0, eps, 100)
```

Out[32]: [array([-0.4972512 , 0.25407859]), 0, 1]

In [33]:
```
# (ii) (x0, y0) = (-1.5, 1.25)

eps = 10e-4

x0_ii = [-1.5, 1.25]

Newton(x0_ii, eps, 100)
```

Out[33]: [array([-0.49725127, 0.25407855]), 0, 5]

In [34]:
```
# (iii) (x0, y0) = (-3, 3)

eps = 10e-4

x0_iii = [-3, 3]

Newton(x0_iii, eps, 100)
```

Out[34]: [array([-0.50247638, -0.24624809]), 0, 8]

> The performance with Newton is much better for (ii) and (iii) as each iteration converges successfully within 10 runs for all three starting points. The closest guess (i) still took one iteration, the second closes (ii) took 5, and the furthest (iii) took 8 iterations to converge within the given tolerance.

2.

In [38]:
```
def evalF(x):

    F = np.zeros(2)

    F[0] = x[0]**2 + x[1]**2 - 4
    F[1] = np.exp(x[0]) + x[1] - 1

    return F
```

In [37]:
```
def evalJ(x):


    J = np.array([[2*x[0], 2*x[1]],
        [np.exp(x[0]), 1]])


    return J
```

In [39]:
```
def LazyNewton(x0,tol,Nmax):

    ''' Lazy Newton = use only the inverse of the Jacobian for initial guess'''
    ''' inputs: x0 = initial guess, tol = tolerance, Nmax = max its'''
    ''' Outputs: xstar= approx root, ier = error message, its = num its'''

    J = evalJ(x0)
    Jinv = inv(J)
    for its in range(Nmax):

       F = evalF(x0)
       x1 = x0 - Jinv.dot(F)

       if (norm(x1-x0) < tol):
           xstar = x1
           ier =0
           return[xstar, ier,its]

       x0 = x1

    xstar = x1
    ier = 1
    return[xstar,ier,its]
```

In [40]:
```
def Broyden(x0,tol,Nmax):
    '''tol = desired accuracy
    Nmax = max number of iterations'''

    '''Sherman-Morrison
   (A+xy^T)^{-1} = A^{-1}-1/p*(A^{-1}xy^TA^{-1})
    where p = 1+y^TA^{-1}Ax'''

    '''In Newton
    x_k+1 = xk -(G(x_k))^{-1}*F(x_k)'''


    '''In Broyden
    x = [F(xk)-F(xk-1)-\hat{G}_k-1(xk-xk-1)
    y = x_k-x_k-1/||x_k-x_k-1||^2'''

    ''' implemented as in equation (10.16) on page 650 of text'''

    '''initialize with 1 newton step'''

    A0 = evalJ(x0)

    v = evalF(x0)
    A = np.linalg.inv(A0)

    s = -A.dot(v)
    xk = x0+s
    for  its in range(Nmax):
       '''(save v from previous step)'''
       w = v
       ''' create new v'''
       v = evalF(xk)
       '''y_k = F(xk)-F(xk-1)'''
       y = v-w;
       '''-A_{k-1}^{-1}y_k'''
       z = -A.dot(y)
       ''' p = s_k^tA_{k-1}^{-1}y_k'''
       p = -np.dot(s,z)
       u = np.dot(s,A)
       ''' A = A_k^{-1} via Morrison formula'''
       tmp = s+z
       tmp2 = np.outer(tmp,u)
       A = A+1./p*tmp2
       ''' -A_k^{-1}F(x_k)'''
       s = -A.dot(v)
       xk = xk+s
       if (norm(s)<tol):
          alpha = xk
          ier = 0
          return[alpha,ier,its]
    alpha = xk
    ier = 1
    return[alpha,ier,its]
```

In [41]:
```
# (i) x = 1, y = 1

eps = 10e-4

x0 = [1, 1]

Newton(x0, eps, 100)
```

Out[41]: [array([-1.81626407, 0.8373678]), 0, 6]

In [42]:
```
LazyNewton(x0, eps, 100)
```

```
/var/folders/nm/zhg7jf3n4gz76nqmv2nbzvrr0000gn/T/ipykernel_95326/2774305835.py:6: RuntimeWarning: overflow encountered in exp
  F[1] = np.exp(x[0]) + x[1] - 1
```

Out[42]: [array([nan, nan]), 1, 99]

In [43]:
```
Broyden(x0, eps, 100)
```

Out[43]: [array([-1.81626453, 0.83736899]), 0, 9]

In [50]:
```
# (ii) x = 1, y = -1
```

```
eps = 10e-4

x0_ii = [1, -1]

Newton(x0_ii, eps, 100)
```

Out[50]: [array([1.00416874, -1.72963729]), 0, 3]

In [51]:
```
LazyNewton(x0_ii, eps, 100)
```

Out[51]: [array([1.00409219, -1.72943086]), 0, 11]

In [46]:
```
Broyden(x0_ii, eps, 100)
```

Out[46]: [array([1.0041689 , -1.72963783]), 0, 3]

In [53]:
```
# (iii) x = 0, y = 0

eps = 10e-4

x0_iii = [0, 0]

Newton(x0_iii, eps, 100)
```

```
---------------------------------------------------------------------------
LinAlgError                               Traceback (most recent call last)
Cell In[53], line 7
      3 eps = 10e-4
      5 x0_iii = [0, 0]
----> 7 Newton(x0_iii, eps, 100)

Cell In[30], line 8, in Newton(x0, tol, Nmax)
      6 for its in range(Nmax):
      7     J = evalJ(x0)
----> 8     Jinv = inv(J)
      9     F = evalF(x0)
     11     x1 = x0 - Jinv.dot(F)

File /opt/miniconda3/envs/earth-analytics-python/lib/python3.11/site-packages/numpy/linalg/_linalg.py:609, in inv(a)
    606 signature = 'D->D' if isComplexType(t) else 'd->d'
    607 with errstate(call=_raise_linalgerror_singular, invalid='call',
    608              over='ignore', divide='ignore', under='ignore'):
--> 609     ainv = _umath_linalg.inv(a, signature=signature)
    610 return wrap(ainv.astype(result_t, copy=False))

File /opt/miniconda3/envs/earth-analytics-python/lib/python3.11/site-packages/numpy/linalg/_linalg.py:104, in _raise_linalgerror_singular(err, flag)
    103 def _raise_linalgerror_singular(err, flag):
--> 104     raise LinAlgError("Singular matrix")

LinAlgError: Singular matrix
```

In [54]:
```
LazyNewton(x0_iii, eps, 100)
```

```
---------------------------------------------------------------------------
LinAlgError                               Traceback (most recent call last)
Cell In[54], line 1
----> 1 LazyNewton(x0_iii, eps, 100)

Cell In[39], line 8, in LazyNewton(x0, tol, Nmax)
      5 ''' Outputs: xstar= approx root, ier = error message, its = num its'''
      7 J = evalJ(x0)
----> 8 Jinv = inv(J)
      9 for its in range(Nmax):
     11     F = evalF(x0)

File /opt/miniconda3/envs/earth-analytics-python/lib/python3.11/site-packages/numpy/linalg/_linalg.py:609, in inv(a)
    606 signature = 'D->D' if isComplexType(t) else 'd->d'
    607 with errstate(call=_raise_linalgerror_singular, invalid='call',
    608              over='ignore', divide='ignore', under='ignore'):
--> 609     ainv = _umath_linalg.inv(a, signature=signature)
    610 return wrap(ainv.astype(result_t, copy=False))

File /opt/miniconda3/envs/earth-analytics-python/lib/python3.11/site-packages/numpy/linalg/_linalg.py:104, in _raise_linalgerror_singular(err, flag)
    103 def _raise_linalgerror_singular(err, flag):
--> 104     raise LinAlgError("Singular matrix")

LinAlgError: Singular matrix
```

In [55]:
```
Broyden(x0_iii, eps, 100)
```

```
---------------------------------------------------------------------------
LinAlgError                               Traceback (most recent call last)
Cell In[55], line 1
----> 1 Broyden(x0_iii, eps, 100)

Cell In[40], line 24, in Broyden(x0, tol, Nmax)
     21 A0 = evalJ(x0)
     23 v = evalF(x0)
---> 24 A = np.linalg.inv(A0)
     26 s = -A.dot(v)
     27 xk = x0+s

File /opt/miniconda3/envs/earth-analytics-python/lib/python3.11/site-packages/numpy/linalg/_linalg.py:609, in inv(a)
    606 signature = 'D->D' if isComplexType(t) else 'd->d'
    607 with errstate(call=_raise_linalgerror_singular, invalid='call',
    608               over='ignore', divide='ignore', under='ignore'):
--> 609     ainv = _umath_linalg.inv(a, signature=signature)
    610 return wrap(ainv.astype(result_t, copy=False))

File /opt/miniconda3/envs/earth-analytics-python/lib/python3.11/site-packages/numpy/linalg/_linalg.py:104, in _raise_linalgerror_singular(err, flag)
    103 def _raise_linalgerror_singular(err, flag):
--> 104     raise LinAlgError("Singular matrix")

LinAlgError: Singular matrix
```

> For (i) [1, 1], LazyNewton fails to converge and gives and overflow warning because the initial Jacobian matrix will exponentially increase as x overshoots. Normal Newton's and Broyden both converge but Newton's takes 6 steps compared to Broyden's 9. For (ii) [1, -1], all three methods converge. Newton's and Broyden's take 3 iterations while Lazy Newton takes 11. All three methods fail for (iii) [0, 0] because the Jacobian at that point has determinant = 0 and is thus non-invertible.

$$det \begin{bmatrix} 2(0) & 2(0) \\ e^{(0)} & 1 \end{bmatrix} = 0(1) - 0(1) = 0$$

3.

In [69]:
```
def evalF(x):

    F = np.zeros(3)

    F[0] = x[0] + np.cos(x[0] * x[1] * x[2]) - 1
    F[1] = (1 - x[0])**(1/4) + x[1] + 0.05*x[2]**2 - 0.15*x[2] - 1
    F[2] = -x[0]**2 - 0.1*x[1]**2 + 0.01*x[0] + x[2] - 1

    return F
```

In [70]:
```
def evalJ(x):


    J = np.array([[1 - np.sin(x[0]*x[1]*x[2]) * (x[1]*x[2]), -np.sin(x[0]*x[1]*x[2]) * x[0]*x[2],
                  -np.sin(x[0]*x[1]*x[2]) * x[0]*x[1]],
        [(1/4)*(1-x[0])**(-3/4), 1, (2*.05)*x[2] - 0.15],
        [-2*x[0], (-2*0.1)*x[1] + 0.01, 1]])


    return J
```

In [73]:
```
def evalg(x):
# The least squares function you are minimizing

    F = evalF(x)
    g = F[0]**2 + F[1]**2 + F[2]**2
    return g
```

In [72]:
```
def eval_gradg(x):
# Grad g
    F = evalF(x)
    J = evalJ(x)

    gradg = np.transpose(J).dot(F)
    return gradg
```

In [87]:
```
def SteepestDescent(x,tol,Nmax):

    for its in range(Nmax):
        g1 = evalg(x)
        z = eval_gradg(x)
        z0 = norm(z)

# below is finding the step-length using the quadratic min.
        if z0 == 0:
            print("zero gradient")
        z = z/z0
        alpha1 = 0
        alpha3 = 1
        dif_vec = x - alpha3*z
        g3 = evalg(dif_vec)

        while g3>=g1:
            alpha3 = alpha3/2
            dif_vec = x - alpha3*z
            g3 = evalg(dif_vec)

        if alpha3<tol:
            print("no likely improvement")
            ier = 0
            return [x,g1,ier, its]

        alpha2 = alpha3/2
```

```
        dif_vec = x - alpha2*z
        g2 = evalg(dif_vec)

        h1 = (g2 - g1)/alpha2
        h2 = (g3-g2)/(alpha3-alpha2)
        h3 = (h2-h1)/alpha3

        alpha0 = 0.5*(alpha2 - h1/h3)
        dif_vec = x - alpha0*z
        g0 = evalg(dif_vec)

        if g0<=g3:
            alpha = alpha0
            gval = g0

        else:
            alpha = alpha3
            gval =g3
#####
        x = x - alpha*z

        if abs(gval - g1)<tol:
            ier = 0
            return [x,gval,ier, its]

    print('max iterations exceeded')
    ier = 1
    return [x,g1,ier, its]
```

In [120…
```
def SteepestNewton(x0, steepest_tol, final_tol, nmax):

    steep_root, steep_g1, steep_ier, steep_its = SteepestDescent(x0, steepest_tol, nmax)

    newt_root, newt_ier, newt_its = Newton(steep_root, final_tol, nmax)

    return[newt_root,newt_ier,newt_its+steep_its]
```

In [121…
```
# farther initial guess examples

eps = 10e-10

x0 = [.99, .5, .2]

Newton(x0, eps, 100)
```

```
/var/folders/nm/zhg7jf3n4gz76nqmv2nbzvrr0000gn/T/ipykernel_95326/1678892762.py:6: RuntimeWarning: invalid value encountered in scalar power
  [(1/4)*(1-x[0])**(-3/4), 1, (2*.05)*x[2] - 0.15],
```

Out[121… `[array([nan, nan, nan]), 1, 99]`

In [107…
```
SteepestDescent(x0, eps, 100)
```

Out[107…
```
[array([4.81630567e-06, 1.00047006e-01, 1.00099972e+00]),
 np.float64(4.1658635466046135e-11),
 0,
 8]
```

In [109…
```
SteepestNewton(x0, 5e-2, eps, 100)
```

Out[109… `[array([1.769473e-17, 1.000500e-01, 1.001001e+00]), 0, 6]`

> The combination of Steepest Descent up to tolerance 5×10^{-2} and Newton's consistently converges in the fewest number of iterations. If the initial guess (ex. [.5, .5, .5]) is within the basin of convergence, Newton's and SteepestNewton's can converge in the same number of steps or even show advantage to pure Newton's, with Steepest Descent only slightly slower. But with a far enough initial guess away from the true root(ex. [.99, .5, .2]), which is more realistic in practice, Newton's can diverge and throw nan errors. In this case, pure Steepest Descent still converges, but passing the progress off to Newton's once close cuts down on the number of iterations needed.

In [118…
```
# close initial guess example

x1 = [.5, .5, .5]

Newton(x1, eps, 100)
```

Out[118… `[array([2.73491321e-17, 1.00050000e-01, 1.00100100e+00]), 0, 5]`

In [117…
```
SteepestDescent(x1, eps, 100)
```

Out[117…
```
[array([1.33760971e-06, 1.00043645e-01, 1.00100077e+00]),
 np.float64(4.639288600948614e-11),
 0,
 7]
```

In [116…
```
SteepestNewton(x1, 5e-2, eps, 100)
```

Out[116… `[array([4.25427364e-17, 1.00050000e-01, 1.00100100e+00]), 0, 6]`